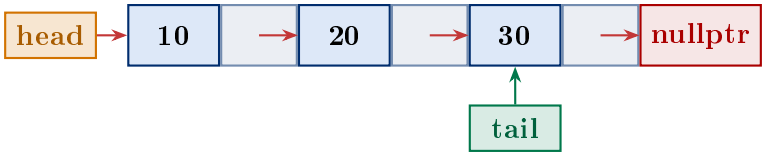
Linked Lists

From Absolute Basics to Advanced Mastery

Complete Study Notes with C++ Implementation



Topics Covered

- Singly Linked List Implementation
- Deletion Methods
- Remove & Re-insert
- Utility Functions
- Reverse & Middle Element
- Doubly Linked List Implementation

github.com/SouvikGhorui/Data-Structure-and-Algorithm-with-CPP

Contents

1 What is a Linked List? 2
- 1.1 The Core Idea . . . 2
- 1.2 Node Class . . . 2
- 1.3 LinkedList Class with head and tail . . . 2

2 Singly Linked List Implementation 4
- 2.1 push_front – Insert at Beginning . . . 4
- 2.2 push_back – Insert at End . . . 4
- 2.3 pop_front – Remove from Beginning . . . 5
- 2.4 pop_back – Remove from End . . . 5
- 2.5 insert – Insert at Any Position . . . 6
- 2.6 show – Display the List . . . 7

3 Utility Functions 8
- 3.1 search – Find a Value . . . 8
- 3.2 length – Count Nodes . . . 8
- 3.3 main() – Build and Query a List . . . 9

4 Deletion Methods 10
- 4.1 pop(x) – Delete Node by Value . . . 10

5 Reverse and Middle Element 12
- 5.1 ReverseLL – Recursive Reverse Print . . . 12
- 5.2 Finding the Middle – Tortoise and Hare . . . 13

6 Remove and Re-insert 15
- 6.1 insert(pos, Node*) – Insert Using a Node Pointer . . . 15
- 6.2 remove_insert – Move a Node from One Position to Another . . . 15

7 Doubly Linked List Implementation 17
- 7.1 Why a Doubly Linked List? . . . 17
- 7.2 Node Class for DLL . . . 17
- 7.3 push_front – DLL . . . 17
- 7.4 push_back – DLL . . . 18
- 7.5 pop_front – DLL . . . 18
- 7.6 pop_back – DLL: now truly O(1) . . . 18
- 7.7 show – DLL . . . 19
- 7.8 Complete DLL main() . . . 19

8 SLL vs DLL – Comparison 20
- 8.1 Operation Complexity . . . 20

9 Advanced Operations **21**
9.1 True List Reversal – Three Pointer Technique 21
9.2 Floyd's Cycle Detection 21
9.3 Merge Sort on Linked List 22

10 Complete Complexity Reference **23**
10.1 All Methods at a Glance 23

1. What is a Linked List?

1.1 The Core Idea

A **Linked List** is a linear data structure made of **nodes**. Each node stores two things:

1. **data** – the actual value you want to store
2. **next** – a pointer to the next node in the chain

The last node's `next` is `nullptr` – meaning nothing follows it.

1.2 Node Class

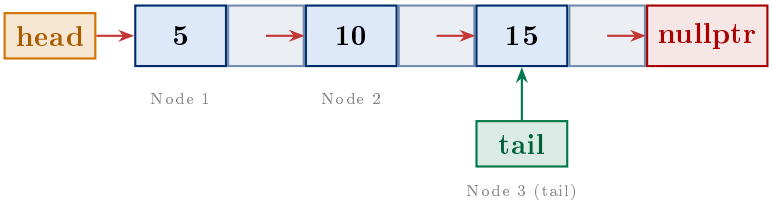
```cpp
class Node {
public:
    int data;     // stores the value
    Node* next;   // pointer to the next node

    Node(int val) {
        data = val;
        next = nullptr;  // C++11 type-safe null pointer
    }
};
```

Listing 1.1: Node class – modern C++11 with nullptr

The code uses `nullptr` (C++11) instead of `NULL` (C-style macro). `nullptr` is type-safe – it can only be assigned to pointer types, preventing subtle bugs that `NULL` (which is just `0`) can cause.

1.3 LinkedList Class with head and tail

Keeping both a `head` and `tail` pointer is a smart design choice that makes `push_back` run in $O(1)$ instead of $O(n)$.




```cpp
class LinkedList {
public:
    Node* head;
    Node* tail;

    LinkedList() {
        head = tail = nullptr;
    }
};
```

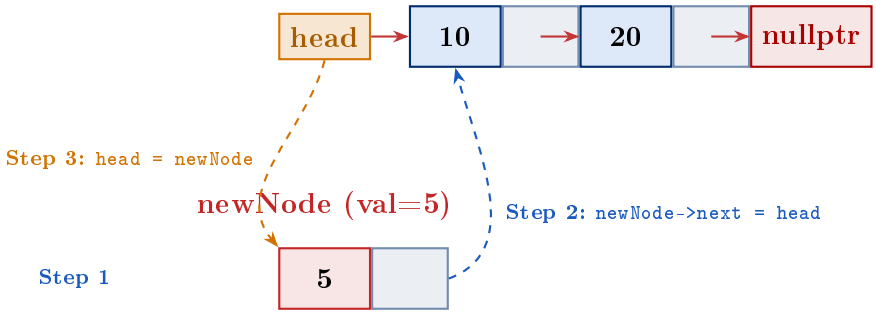
Listing 1.2: LinkedList class skeleton

Operation	Only head	head + tail
`push_front`	$O(1)$	$O(1)$
`push_back`	$O(n)$ – must walk to end	$O(1)$ – tail is direct
`pop_back`	$O(n)$	$O(n)$ – no prev pointer in SLL
`show`	$O(n)$	$O(n)$

2. Singly Linked List Implementation

2.1 push_front – Insert at Beginning

Three steps: (1) Create new node (2) Link new node to old head (3) Update head



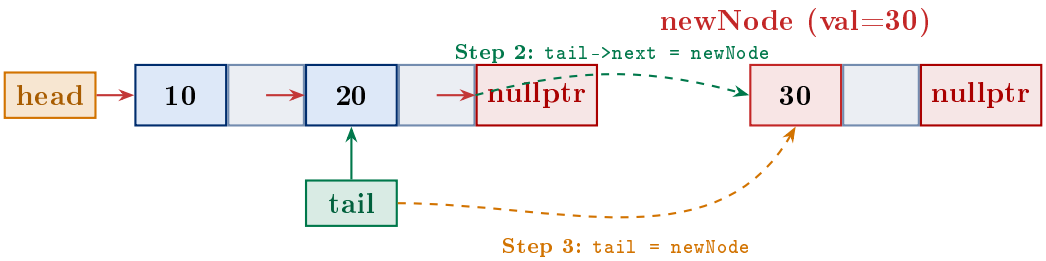

```
void push_front(int val) {
    // Step 1: create new node
    Node* newNode = new Node(val);

    if (head == nullptr) {
        head = tail = newNode;    // empty list: both pointers point here
        return;
    }
    // Step 2: link new node to old head
    newNode->next = head;
    // Step 3: update head to new node
    head = newNode;
}
```

Listing 2.1: push_front – O(1)

Time: $O(1)$ – no traversal, just pointer updates.
Space: $O(1)$ – one new node only.

2.2 push_back – Insert at End

Three steps: (1) Create new node (2) Link old tail to new node (3) Update tail




```cpp
void push_back(int val) {
    // Step 1: create new node
    Node* newNode = new Node(val);

    if (head == nullptr) {
        head = tail = newNode;    // empty list
        return;
    }
    // Step 2: link old tail to new node
    tail->next = newNode;
    // Step 3: update tail to new node
    tail = newNode;
}
```

Listing 2.2: push_back – O(1) with tail pointer

Because we store a `tail` pointer, `push_back` is $O(1)$. Without `tail`, we would have to walk the entire list each time – $O(n)$.

2.3 pop_front – Remove from Beginning

```cpp
void pop_front() {
    if (head == nullptr) {
        cout << "List is empty!" << endl;
        return;
    }
    Node* temp = head;        // save old head
    head = head->next;        // advance head
    temp->next = nullptr;     // disconnect
    delete temp;              // free memory
}
```

Listing 2.3: pop_front – O(1)

2.4 pop_back – Remove from End

```cpp
void pop_back() {
    if (head == nullptr) {
```

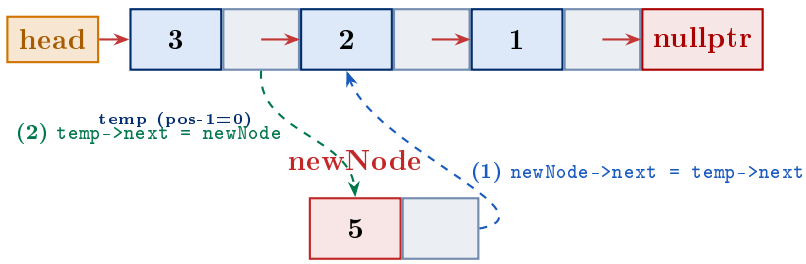
```
        cout << "List is empty!" << endl;
        return;
    }
    if (head == tail) {     // only one node
        delete head;
        head = tail = nullptr;
        return;
    }
    Node* temp = head;
    while (temp->next != tail) {    // walk to second-to-last node
        temp = temp->next;
    }
    delete tail;
    tail = temp;
    tail->next = nullptr;
}
```

Listing 2.4: pop_back – O(n) for singly linked list

Why is `pop_back` $O(n)$ for SLL even with a tail pointer? We know where `tail` is but must find the node *before* it to update its `next` to `nullptr`. There is no `prev` pointer in a singly linked list, so we must walk from `head`. This is exactly what the Doubly Linked List (Chapter 6) solves – `pop_back` becomes $O(1)$.

2.5 insert – Insert at Any Position

insert(5, pos=1) into list [3] -> [2] -> [1] -> nullptr




```
void insert(int val, int pos) {
    if (pos < 0) return;
    if (pos == 0) { push_front(val); return; }

    Node* newNode = new Node(val);
    Node* temp = head;
    for (int i = 0; i < pos - 1; i++) {
        if (temp == nullptr) { cout << "Invalid position"; return ; }
        temp = temp->next;
    }
```

```
    // (1) newNode points to what temp currently points to
    newNode->next = temp->next;
    // (2) temp now points to newNode
    temp->next = newNode;
}
```

Listing 2.5: insert at position – O(n)

2.6 show – Display the List

```
void show() const {   // const: does not modify the list
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "nullptr" << endl;
}
// Output: 3 -> 5 -> 2 -> 1 -> nullptr
```

Listing 2.6: show – const, O(n)

3. Utility Functions

3.1 search – Find a Value

```
bool search(Node* head, int x) const {
    Node* temp = head;
    while (temp != nullptr) {
        if (temp->data == x) return true;
        temp = temp->next;
    }
    return false;
}
```

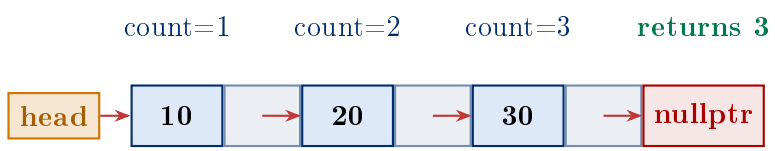
Listing 3.1: search – fixed, const, O(n)

This is a clean, correct implementation. Note it is marked `const` because it only reads the list – it does not modify any nodes. The `bool` return makes it easy to use in an `if` condition: `if (LL.search(LL.head, 4)) { ... }`

What was wrong in the original – and why it is fixed here

The original `search` in `singlyLinkedList.cpp` had a critical bug: `int count = 0;` was declared *inside* the `while` loop. This reset `count` to zero on every iteration, so the function always reported position 0. The clean version above uses `bool` return and does not attempt to track position, avoiding that problem entirely.

3.2 length – Count Nodes




```
int length(Node* head) const {
    Node* temp = head;
    int count = 0;
    while (temp != nullptr) {
        count++;
        temp = temp->next;
```

```
    }
    return count;
}
```

Listing 3.2: length – const, O(n)

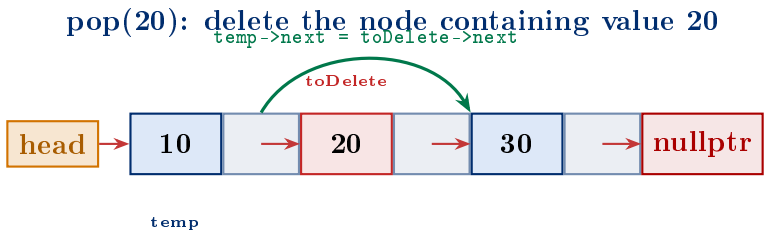
3.3 main() – Build and Query a List

```
int main() {
    int n, val;
    cout << "Enter number of elements: ";
    cin >> n;

    LinkedList LL;
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> val;
        LL.push_back(val);
    }

    if (LL.search(LL.head, 4))
        cout << "4 found in list" << endl;

    LL.show();
    cout << "Total nodes = " << LL.length(LL.head) << endl;
    return 0;
}
```

Listing 3.3: Creating a list and using utility functions

4. Deletion Methods

4.1 pop(x) – Delete Node by Value




```
void pop(int x) {
    if (head == nullptr) return;

    // Special case: value is at head
    if (head->data == x) {
        pop_front();
        return;
    }

    Node* temp = head;
    // FIXED: check nullptr FIRST, then access ->data (short-circuit)
    while (temp->next != nullptr && temp->next->data != x) {
        temp = temp->next;
    }

    if (temp->next == nullptr) return;  // value not found

    // Special case: value is at tail
    if (temp->next == tail) {
        pop_back();
        return;
    }

    // General case: bypass and delete
    Node* toDelete = temp->next;
    temp->next = toDelete->next;
    delete toDelete;
}
```

Listing 4.1: pop(x) – fixed: NULL check before data access

Why the original condition was dangerous:
The original code had:

```
while ((temp->next->data != x) && (temp->next != nullptr))
```

C++ evaluates conditions left-to-right. When `temp->next` is `nullptr`, it still evaluates `temp->next->data` first – dereferencing a null pointer and crashing with a segmentation fault.
The fix is simply to swap the order:

```
while (temp->next != nullptr && temp->next->data != x)
// ^^^ nullptr check FIRST -- if false, right side is never
    evaluated
```

5. Reverse and Middle Element

5.1 ReverseLL – Recursive Reverse Print

```
void ReverseLL(Node* head) {
    if (head == nullptr) return;      // base case
    ReverseLL(head->next);            // recurse to the end first
    cout << head->data << " ";        // print on the way BACK up
}
```

Listing 5.1: ReverseLL – prints in reverse, O(n) time O(n) stack

Call stack for list 1 -> 2 -> 3 -> nullptr:

Input: 1 -> 2 -> 3 -> nullptr

Frame 1: ReverseLL(node=1) – recurses into ReverseLL(node=2)

Frame 2: ReverseLL(node=2) – recurses into ReverseLL(node=3)

Frame 3: ReverseLL(node=3) – recurses into ReverseLL(nullptr)

Frame 4: ReverseLL(nullptr) – BASE CASE, returns immediately

Unwinding: Frame 3 prints 3, Frame 2 prints 2, Frame 1 prints 1
Final output: 3 2 1

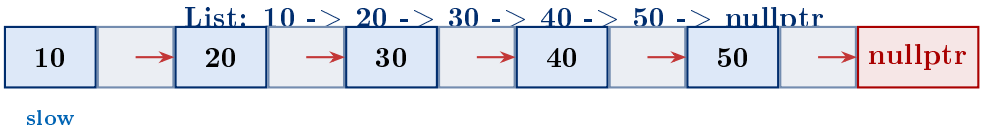
`ReverseLL()` only *prints* in reverse order. It does not modify the list. The original list remains 1 -> 2 -> 3 after the call. To actually reverse the pointers, see Chapter 8.

Time: $O(n)$ – every node visited exactly once.
Space: $O(n)$ – one call-stack frame is pushed per node.

5.2 Finding the Middle – Tortoise and Hare

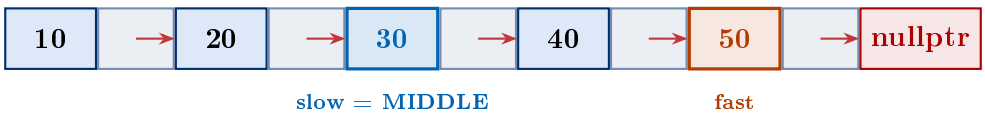
The single-pass **fast/slow pointer** technique (also known as Tortoise and Hare) finds the middle node in one traversal with no length calculation.

How it works: slow advances 1 step per iteration; fast advances 2 steps. When fast hits the end, slow is at the middle.



Iteration	slow (1 step)	fast (2 steps)	fast==nullptr?
Start	node(10)	node(10)	No
1	node(20)	node(30)	No
2	**node(30)**	node(50)	No
3 (stop)	**node(30)** – MIDDLE	fast->next == nullptr	**YES – STOP**

When fast stops, slow is at the middle (node 30):




```cpp
void middleEle(Node* head) const {
    if (head == nullptr) return;

    Node* slow = head;
    Node* fast = head;

    // fast moves 2 steps; slow moves 1 step
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;          // tortoise: 1 step
        fast = fast->next->next;    // hare: 2 steps
    }
    // when fast hits the end, slow is at the middle
    cout << "Middle element = " << slow->data << endl;
}
// For list 10->20->30->40->50: output = 30
// For list 10->20->30->40:     output = 20
```

Listing 5.2: middleEle – single-pass fast/slow pointer

Complexity Analysis

Time: $O(n/2)$ – fast pointer reaches the end in half a traversal.
Space: $O(1)$ – only two extra pointers regardless of list size.
vs two-pass: $O(n)$ for length + $O(n/2)$ walk = $1.5n$ operations. Single-pass does

$n/2$ – twice as efficient in practice.

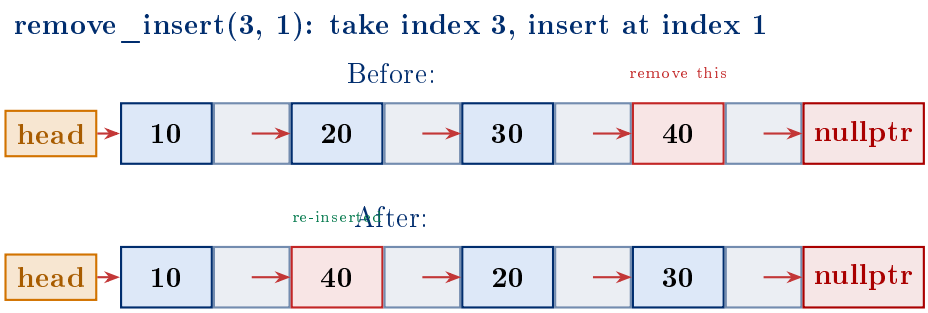
6. Remove and Re-insert

6.1 insert(pos, Node*) – Insert Using a Node Pointer

```
void insert(int pos, Node* node) {
    if (pos == 0) { push_front(node->data); return; }

    int count = 0;
    Node* temp = head;
    while (count != pos - 1) {
        temp = temp->next;
        count++;
    }
    node->next = temp->next;
    temp->next = node;
}
```

Listing 6.1: insert using a node pointer

6.2 remove_insert – Move a Node from One Position to Another




```
void remove_insert(int remove_pos, int insert_pos) {
    Node* temp = head;
    int count = 0;

    // Step 1: navigate to node BEFORE remove_pos
    if (remove_pos == 0) {
        // FIXED: save old head before overwriting it
        Node* oldHead = head;
        head = head->next;
```

```
        // Create a new standalone node with that value
        Node* moved = new Node(oldHead->data);
        delete oldHead;          // free memory (was missing before)
        insert(insert_pos, moved);
        return;
    }

    while (count != remove_pos - 1) {
        temp = temp->next;
        count++;
    }

    // Step 2: copy value, unlink original
    Node* moved = new Node(temp->next->data);
    Node* old   = temp->next;
    temp->next  = old->next;
    delete old;              // free the unlinked node

    // Step 3: insert the copy at desired position
    insert(insert_pos, moved);
}
```

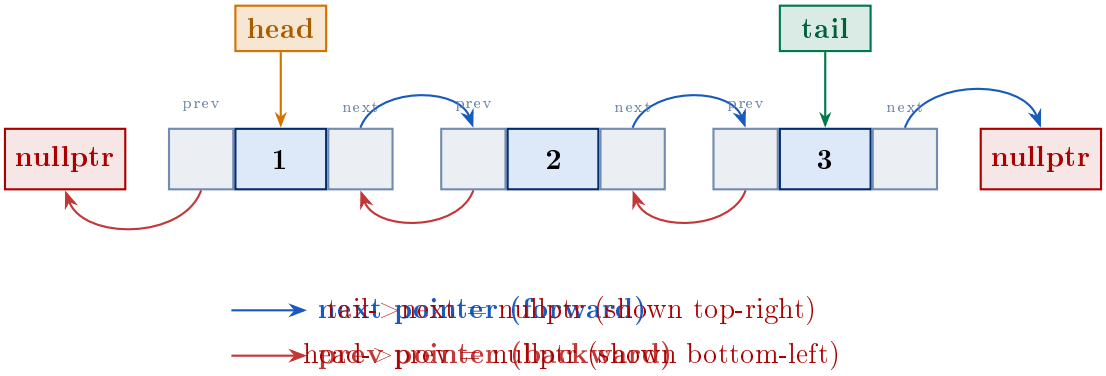
Listing 6.2: remove_insert – fixed memory leak at pos 0

Memory leak in the original `remove_insert`: When `remove_pos == 0` the original code ran `head = head->next;` without saving the old head pointer first. The old head node became unreachable – its memory was never freed, producing a leak. The fixed version saves `oldHead`, then `deletes` it after copying the value into `moved`.

7. Doubly Linked List Implementation

7.1 Why a Doubly Linked List?

A doubly linked list adds a `prev` pointer to every node. This single addition makes `pop_back` drop from $O(n)$ to $O(1)$ and enables efficient backward traversal.



7.2 Node Class for DLL

```
class Node {
public:
    int   data;
    Node* next;
    Node* prev;      // backward pointer -- the key difference from SLL

    Node(int val) {
        data = val;
        next = nullptr;
        prev = nullptr;
    }
};
```

Listing 7.1: DLL Node with prev and next

7.3 push_front – DLL

```
void push_front(int val) {
    Node* newNode = new Node(val);
```

```
    if (head == nullptr) {
        head = tail = newNode;
    } else {
        newNode->next = head;      // new node -> old head
        head->prev    = newNode;   // old head <- new node
        head          = newNode;   // update head
    }
}
```

Listing 7.2: DLL push_front – O(1)

7.4 push_back – DLL

```
void push_back(int val) {
    Node* newNode = new Node(val);
    if (head == nullptr) {
        head = tail = newNode;
    } else {
        newNode->prev = tail;      // new node <- old tail
        tail->next    = newNode;   // old tail -> new node
        tail          = newNode;   // update tail
    }
}
```

Listing 7.3: DLL push_back – O(1)

7.5 pop_front – DLL

```
void pop_front() {
    if (head == nullptr) return;
    Node* temp = head;
    head = head->next;
    if (head != nullptr)
        head->prev = nullptr;  // new head has no predecessor
    temp->next = nullptr;
    delete temp;
}
```

Listing 7.4: DLL pop_front – O(1)

7.6 pop_back – DLL: now truly O(1)

```
void pop_back() {
    if (head == nullptr) return;
    Node* temp = tail;
    tail = tail->prev;              // jump to second-to-last
    instantly
    if (tail != nullptr)
```

```
        tail->next = nullptr;       // new tail has no successor
    temp->prev = nullptr;
    delete temp;
}
```

Listing 7.5: DLL pop_back – O(1) thanks to prev pointer

This is the defining advantage of DLL. In SLL, `pop_back` is $O(n)$ because there is no way to reach the second-to-last node without a full traversal. In DLL, `tail->prev` gives us that node immediately in $O(1)$.

7.7 show – DLL

```
void show() const {
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data << "<=>";
        temp = temp->next;
    }
    cout << "nullptr" << endl;
}
// Output: 1<=>2<=>3<=>nullptr
```

Listing 7.6: DLL show – const, O(n)

7.8 Complete DLL main()

```
int main() {
    doublyList dll;
    dll.push_back(1);
    dll.push_back(2);
    dll.push_back(3);
    dll.pop_back();      // removes 3 in O(1)
    dll.show();          // Output: 1<=>2<=>nullptr
    return 0;
}
```

Listing 7.7: main() for Doubly Linked List

8. SLL vs DLL – Comparison

8.1 Operation Complexity

Operation	SLL	DLL	Reason
`push_front`	$O(1)$	$O(1)$	Same logic
`push_back`	$O(1)$	$O(1)$	Both keep tail pointer
`pop_front`	$O(1)$	$O(1)$	Same logic
`pop_back`	$O(n)$	$O(1)$	DLL has prev pointer
`insert`	$O(n)$	$O(n)$	Must search
`search`	$O(n)$	$O(n)$	Must traverse
`show`	$O(n)$	$O(n)$	Full traversal
Memory per node	1 ptr	2 ptrs	DLL costs more RAM

9. Advanced Operations

9.1 True List Reversal – Three Pointer Technique

`ReverseLL` only prints in reverse. This version actually flips the pointers and modifies the list:

```
void reverseList() {
    Node* prev = nullptr;
    Node* curr = head;
    Node* next = nullptr;
    tail = head;            // old head becomes the new tail

    while (curr != nullptr) {
        next       = curr->next;    // save next before overwriting
        curr->next = prev;          // flip pointer
        prev       = curr;          // advance prev
        curr       = next;          // advance curr
    }
    head = prev;            // old tail is the new head
}
```

Listing 9.1: Iterative reverse – modifies the list, O(n) time O(1) space

Dry run on 1 → 2 → 3 → nullptr:

Step	prev	curr	next	Action
Start	nullptr	1	–	initialise
1	nullptr	1	2	1->nullptr; prev=1; curr=2
2	1	2	3	2->1; prev=2; curr=3
3	2	3	nullptr	3->2; prev=3; curr=nullptr
End	3	nullptr	–	head = 3

9.2 Floyd's Cycle Detection

```
bool hasCycle() const {
    Node* slow = head;
    Node* fast = head;
    while (fast != nullptr && fast->next != nullptr) {
```

```
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
```

Listing 9.2: Detect cycle – O(n) time O(1) space

9.3 Merge Sort on Linked List

```
Node* getMiddle(Node* head) {
    Node* slow = head;
    Node* fast = head->next;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}

Node* merge(Node* a, Node* b) {
    if (a == nullptr) return b;
    if (b == nullptr) return a;
    if (a->data <= b->data) {
        a->next = merge(a->next, b); return a;
    }
    b->next = merge(a, b->next); return b;
}

Node* mergeSort(Node* head) {
    if (head == nullptr || head->next == nullptr) return head;
    Node* mid        = getMiddle(head);
    Node* secondHalf = mid->next;
    mid->next        = nullptr;
    return merge(mergeSort(head), mergeSort(secondHalf));
}
// Usage: LL.head = mergeSort(LL.head);
```

Listing 9.3: Merge Sort – O(n log n)

10. Complete Complexity Reference

10.1 All Methods at a Glance

Method	Chapter	Time	Space
`push_front`	SLL Impl.	$O(1)$	$O(1)$
`push_back`	SLL Impl.	$O(1)$	$O(1)$
`pop_front`	SLL Impl.	$O(1)$	$O(1)$
`pop_back` SLL	SLL Impl.	$O(n)$	$O(1)$
`pop_back` DLL	DLL Impl.	$O(1)$	$O(1)$
`insert`	SLL Impl.	$O(n)$	$O(1)$
`pop(x)`	Deletion	$O(n)$	$O(1)$
`search`	Utility	$O(n)$	$O(1)$
`length`	Utility	$O(n)$	$O(1)$
`ReverseLL`	Reverse & Mid	$O(n)$	$O(n)$
`middleEle`	Reverse & Mid	$O(n/2)$	$O(1)$
`remove_insert`	Remove & Insert	$O(n)$	$O(1)$
`show`	All chapters	$O(n)$	$O(1)$
`reverseList`	Advanced	$O(n)$	$O(1)$
`mergeSort`	Advanced	$O(n \log n)$	$O(\log n)$